# TenantPro: Architecture, Implementation and Current State

Oct 30, 2026 · @Anush Kulal M

A technical report on TenantPro, a two-sided rent-management system for the Indian residential rental market, covering its architecture, implemented feature set, payment model, security design, verification results and remaining scope.

## Abstract

TenantPro is a working two-sided rent-management system, deployed and in use, comprising a React Native client (Expo SDK 54) and a Node/Express API over MySQL. It spans 38,251 lines across 123 source files: 14,444 in the backend, 21,609 in the redesigned mobile client, and 2,198 in eight test suites. The API exposes 87 endpoints over 22 tables; the client has 28 screens.

The system's defining design decision is that **money never passes through it**. Rent moves from tenant to landlord directly — by UPI, or through a payment gateway connected to the landlord's own merchant account. This keeps the operator outside the RBI payment-aggregator perimeter, which requires authorisation and a ₹25 crore net-worth floor, and means the platform bills nobody and holds nobody's funds.

Because the application cannot observe money it does not touch, it is built on an explicit confirmation model: a payment is real only once something other than the tenant vouches for it. Two vouchers exist — the landlord tapping Confirm, and a payment gateway asked directly, server-to-server. Both settle through one function, so they cannot disagree about when a rent cycle clears.

The current build is verified by 386 assertions across six suites, including 168 driven end-to-end against a live server and a real MySQL instance, and 71 driving the compiled application in a browser engine. This report documents the architecture, the implemented feature set with interface evidence, the payment and session-security models, the verification methodology, and the work identified but not built.

## 1. Introduction

Indian residential tenancy is largely administered on paper, in WhatsApp threads, and in the landlord's memory. Rent is collected by UPI transfer or cash; the record of it is a screenshot. A tenant who has paid has no receipt the landlord acknowledges, and a landlord with several rooms has no reliable view of who is overdue.

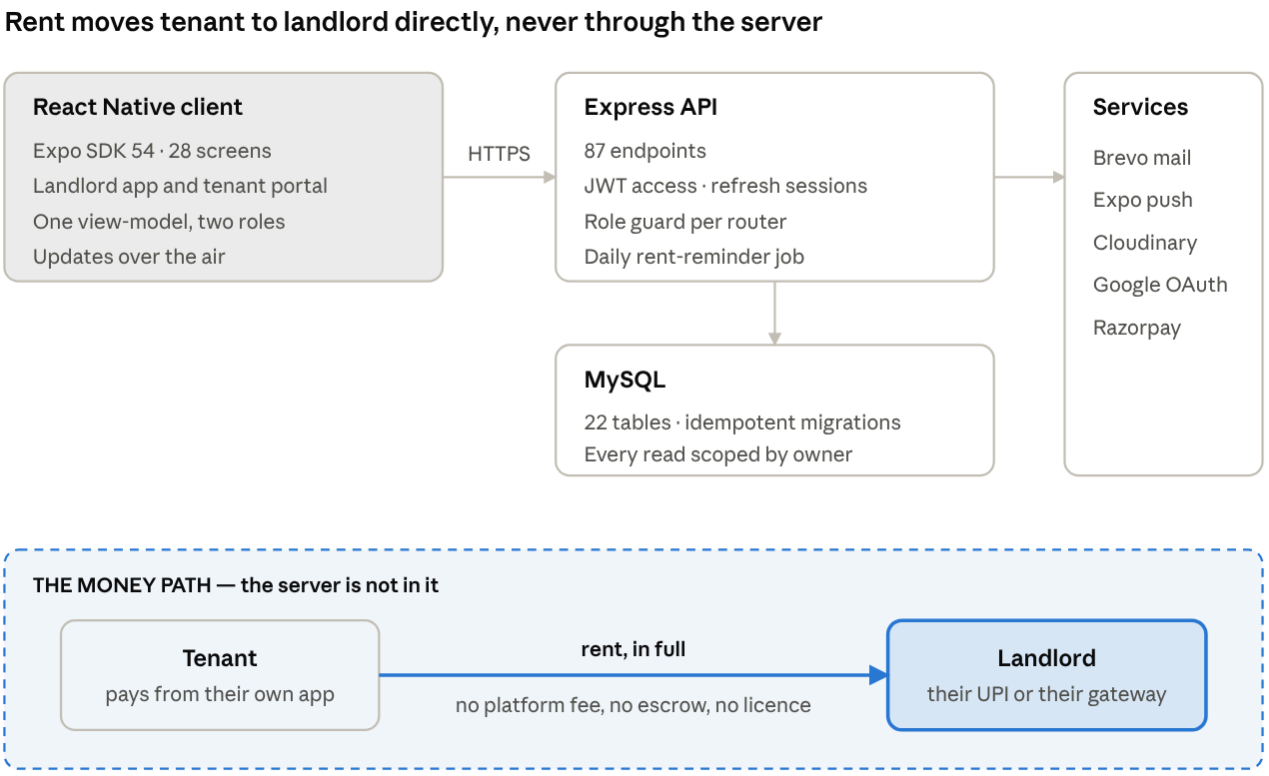
TenantPro addresses the operational layer of that relationship rather than the legal one. It gives a landlord a roster of properties, rooms and tenants with due dates derived from each tenancy's own anniversary, and gives the tenant a parallel view of the same records — what they owe, when, what they have paid, and what their landlord has acknowledged.

Three constraints shaped the implementation more than any feature request.

1. **The application cannot see money.** It has no view of a bank account, so it cannot assert that a transfer occurred. Every payment is therefore a *claim* until something external vouches for it. This is a correctness constraint, not a limitation to be engineered away.
2. **Collecting rent centrally is a regulated activity.** Pooling tenants' rent into one account and paying it out is payment aggregation under RBI rules. The architecture routes money tenant-to-landlord precisely to stay outside that perimeter.
3. **Native code cannot ship over the air.** The client updates through `expo-updates`, which can replace JavaScript but not native modules. Features requiring a native SDK would freeze the release cadence of everything around them, so they are declined in favour of designs that ship like any other change.

## 2. System architecture

The server mediates every piece of information about a tenancy and no part of the payment itself. The dashed band below is the money path: it touches the tenant and the landlord and nothing else.




TenantPro architecture · client, API, data, services, and the money path outside all of them

The client is a single React Native codebase serving both roles. State is derived through one pure function, `deriveVm(state, api)`, which maps server data to everything the screens render; this is what makes the interface testable without a device, and is exercised directly by one of the suites in Section 7.

The API is organised as routers with a role guard mounted once per router rather than per handler. Landlord and tenant accounts live in separate tables with independent auto-increments, so identity alone is not authorisation — `requireOwner` rejects a tenant token on a landlord route, and tenant handlers scope every query through the `tenant_users` to `tenants` link.

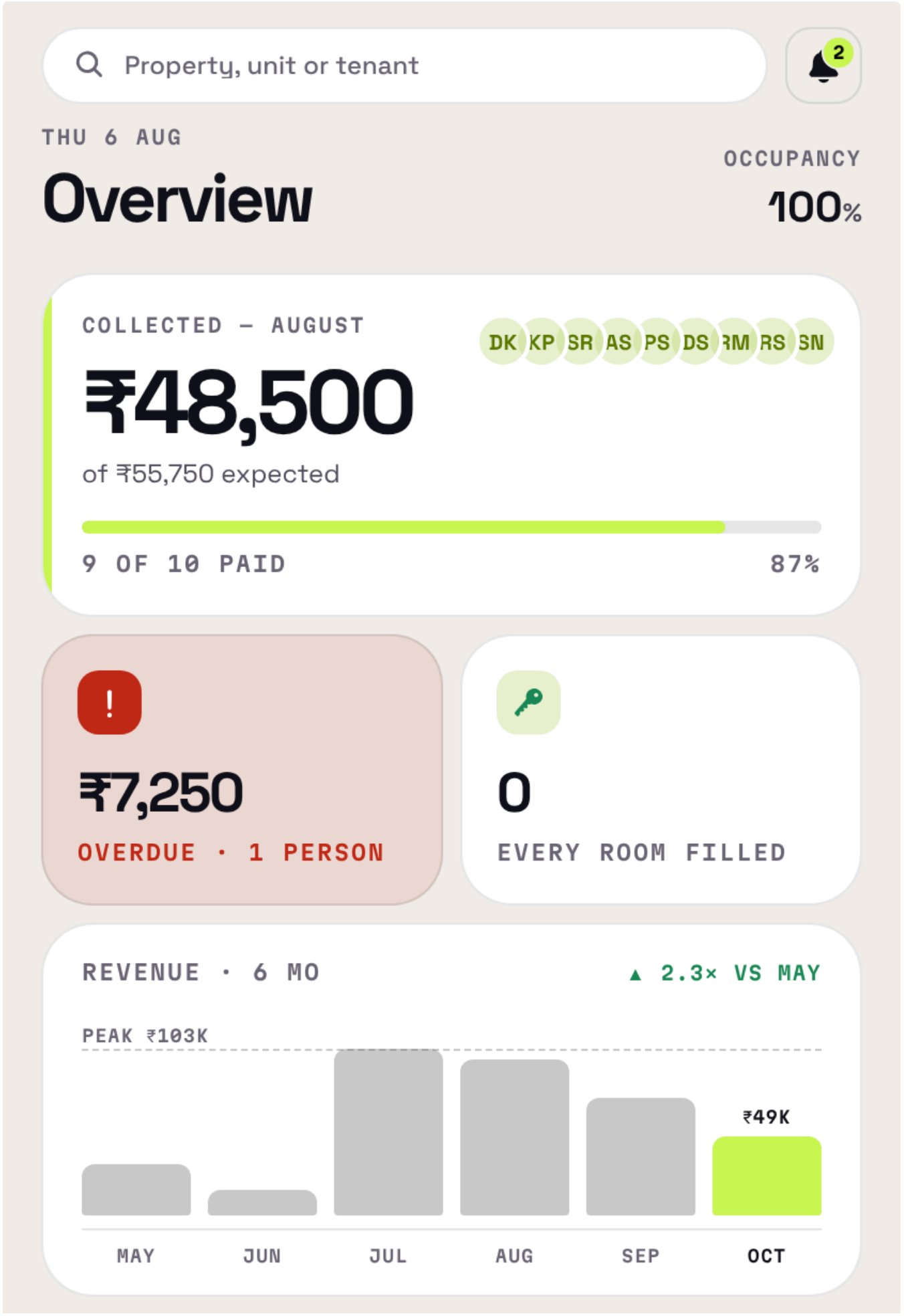
Schema changes apply idempotently at boot: `schema.sql` runs `CREATE TABLE IF NOT EXISTS` on every start, and column-level migrations run through `ensureColumn` and `ensureIndex` guards. A deploy never requires a manual migration step.

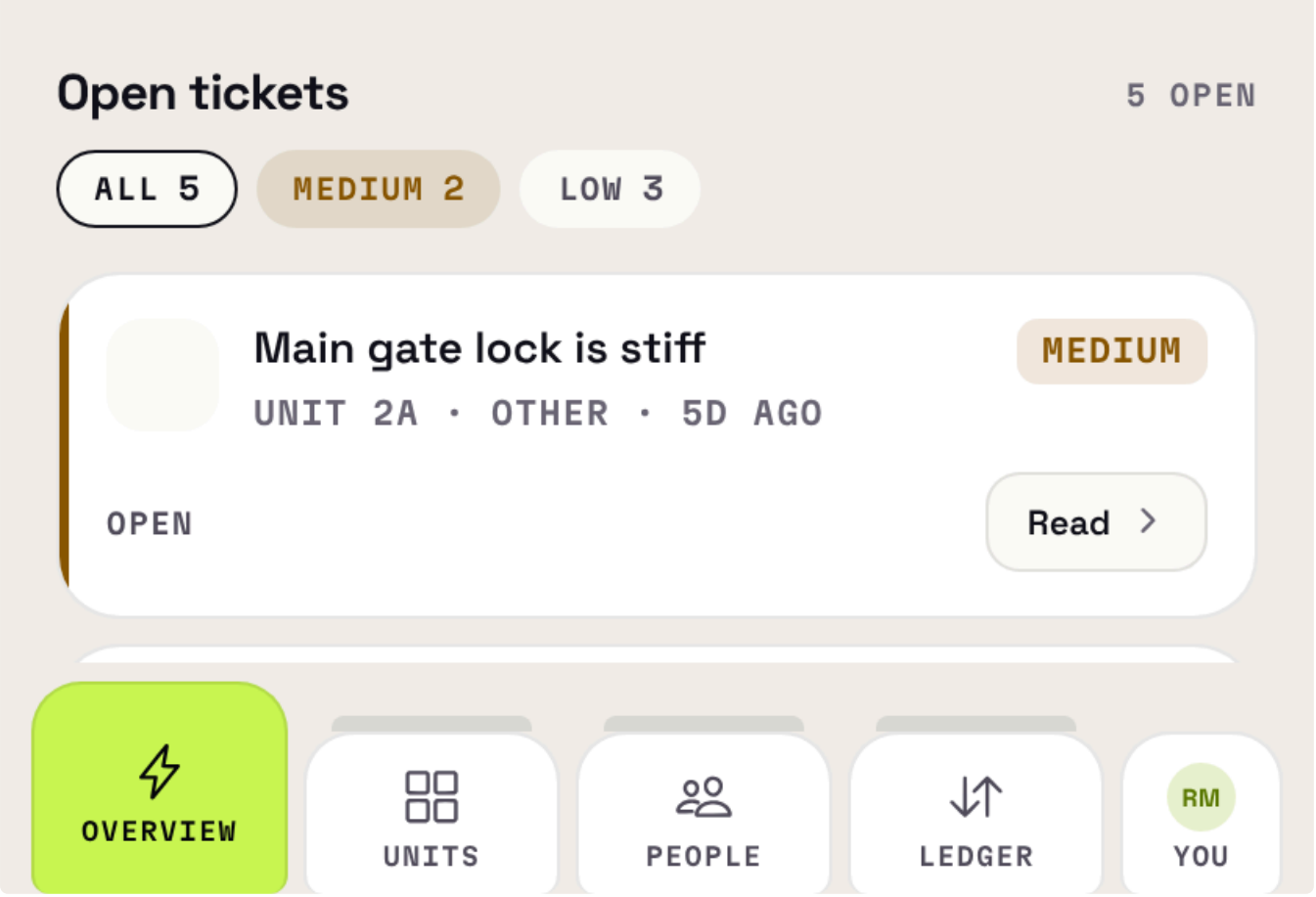
## 3. Landlord-side implementation

All screenshots below are of the running application against a seeded database of three properties, six rooms and ten tenants. No figure in them is placeholder text.

## 3.1 Portfolio overview

The landing screen answers the two questions a landlord opens the app for: who has paid, and who has not.

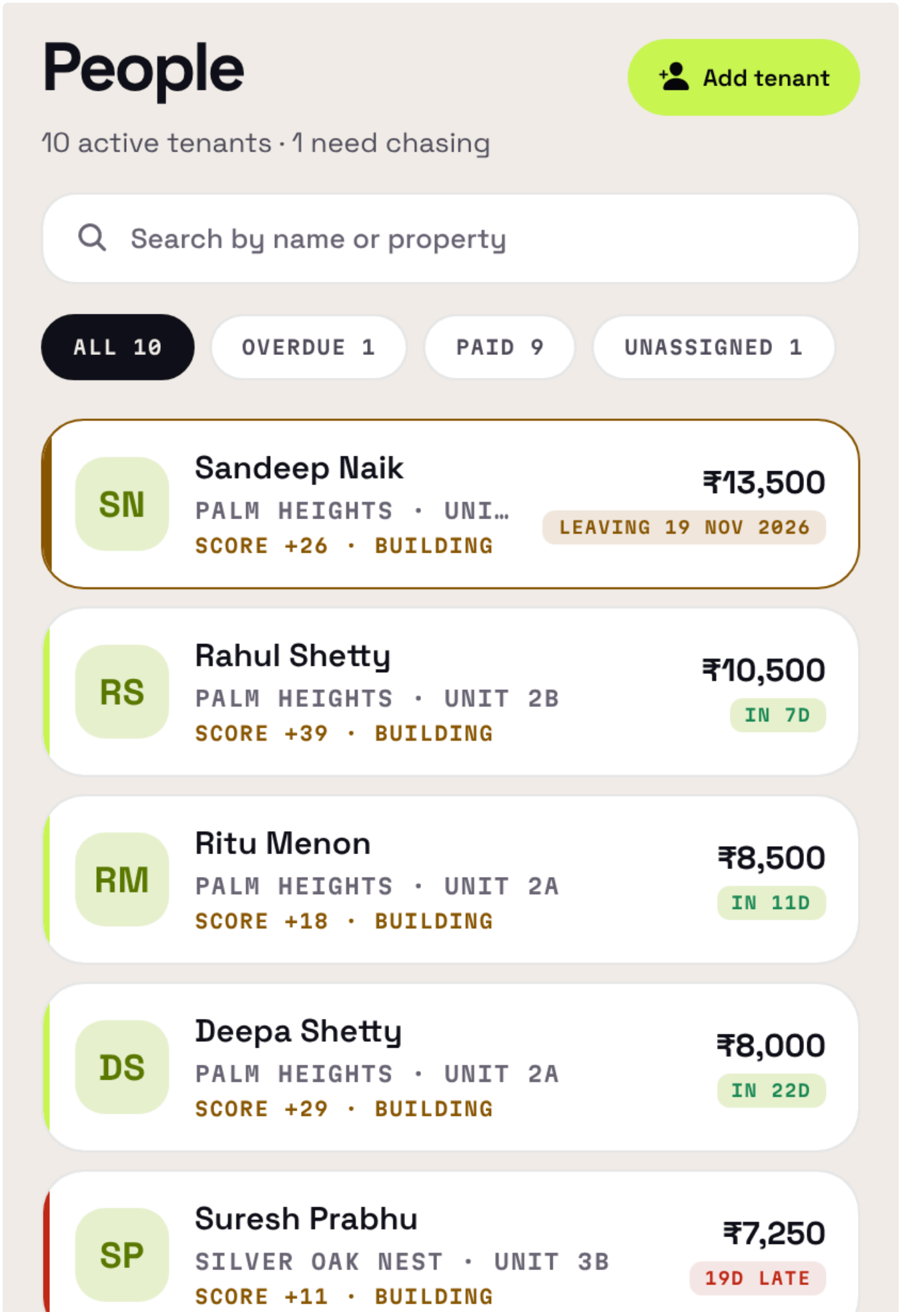

Property, unit or tenant
2
THU 6 AUG
Overview
OCCUPANCY
100%
COLLECTED — AUGUST
DK KP SR AS PS DS RM RS SN
₹48,500
of ₹55,750 expected
9 OF 10 PAID
87%
!
₹7,250
OVERDUE · 1 PERSON
0
EVERY ROOM FILLED
REVENUE · 6 MO
▲ 2.3× VS MAY
PEAK ₹103K
₹49K
MAY
JUN
JUL
AUG
SEP
OCT

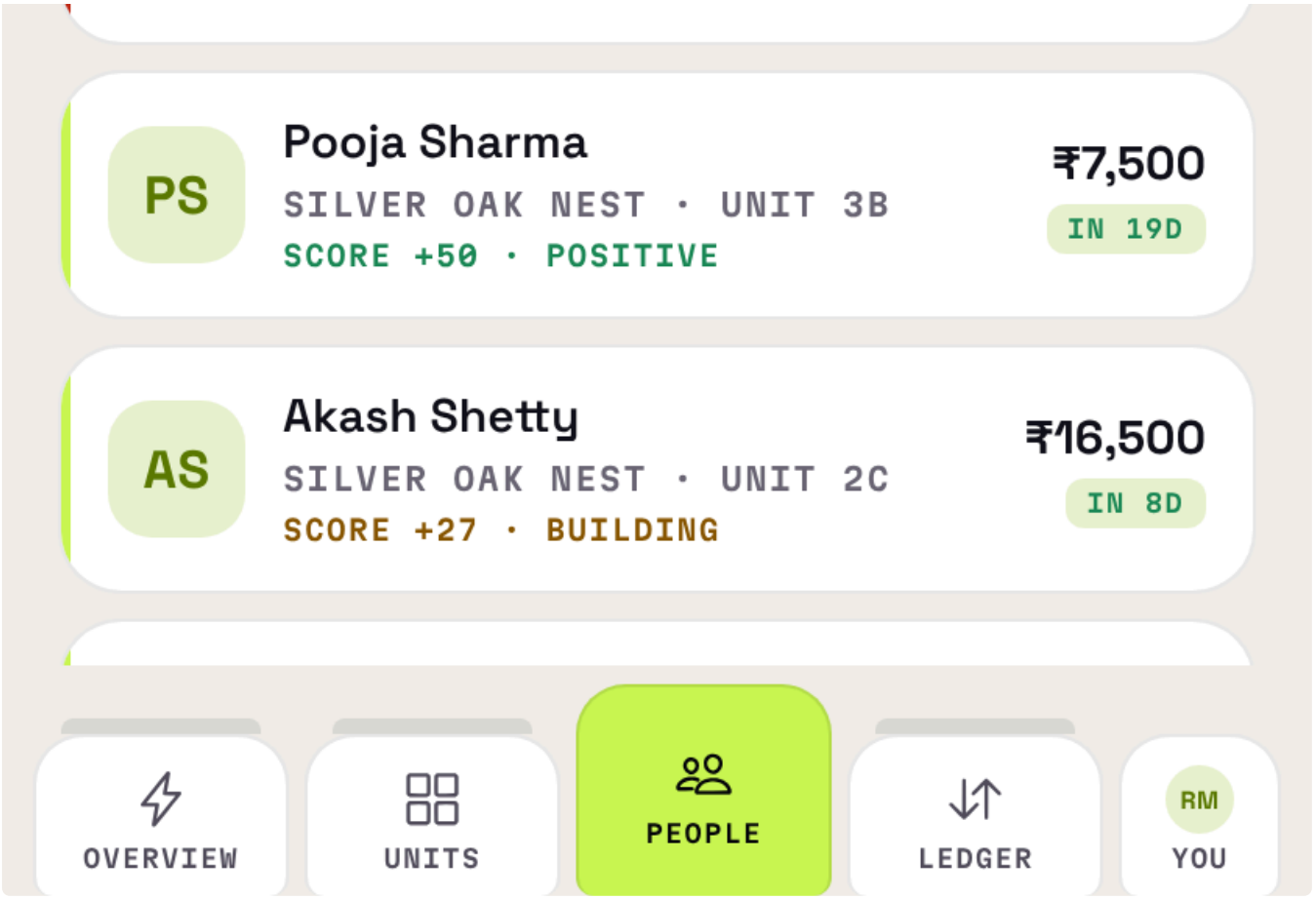



Collection is shown against expectation rather than as a bare total — ₹48,500 of ₹55,750, nine of ten paid — because a number with no denominator cannot be acted on. Overdue is given its own card with a person count. The revenue series is derived from confirmed payments only; declared-but-unconfirmed claims are deliberately excluded, since counting them would overstate income on the strength of a tenant's assertion.

The bell carries a live count of things requiring a decision: declared payments awaiting confirmation, and join requests.

## 3.2 The tenant roster

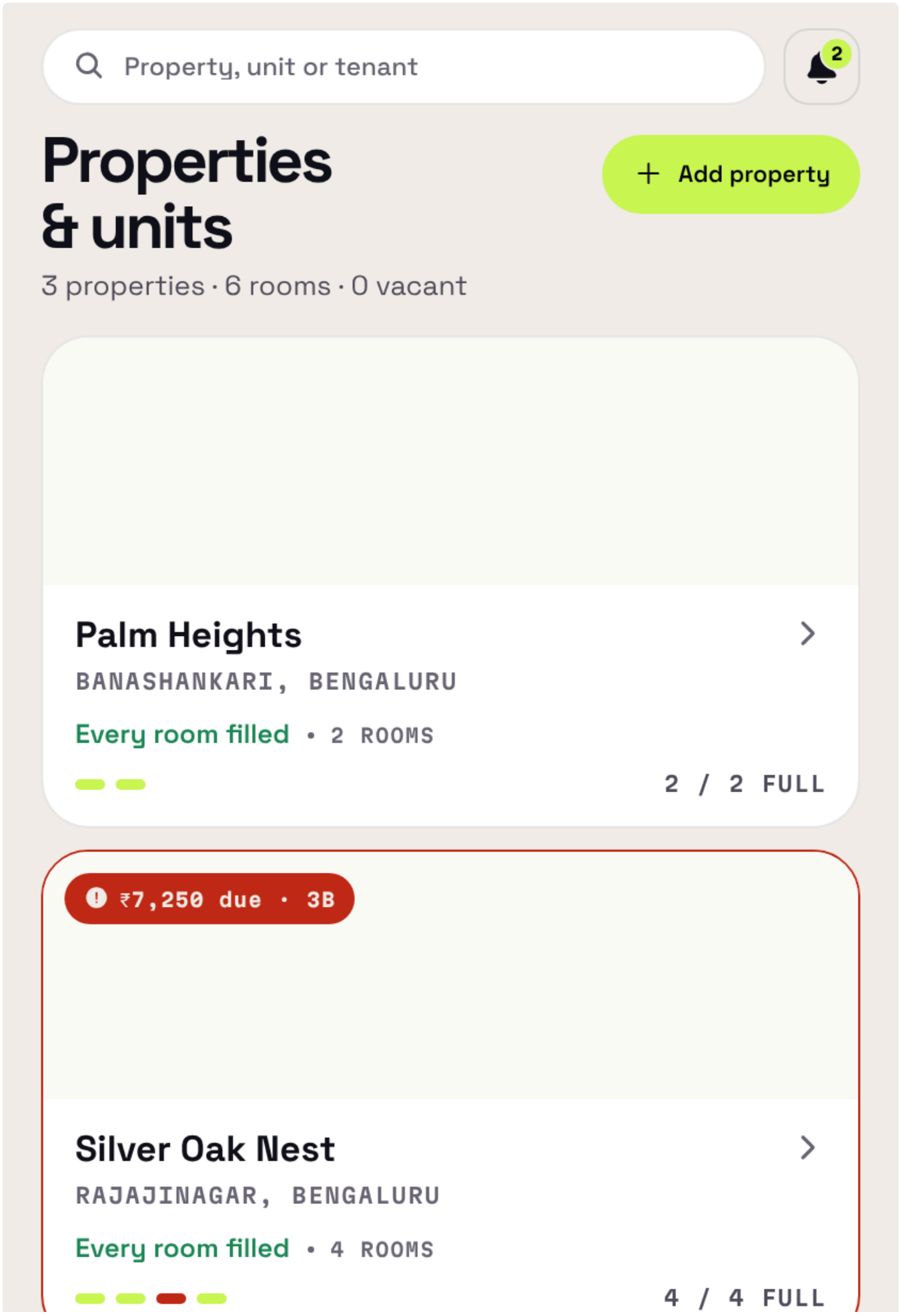

People
Add tenant
10 active tenants · 1 need chasing
Search by name or property
ALL 10
OVERDUE 1
PAID 9
UNASSIGNED 1
SN
Sandeep Naik
PALM HEIGHTS · UNI…
SCORE +26 · BUILDING
₹13,500
LEAVING 19 NOV 2026
RS
Rahul Shetty
PALM HEIGHTS · UNIT 2B
SCORE +39 · BUILDING
₹10,500
IN 7D
RM
Ritu Menon
PALM HEIGHTS · UNIT 2A
SCORE +18 · BUILDING
₹8,500
IN 11D
DS
Deepa Shetty
PALM HEIGHTS · UNIT 2A
SCORE +29 · BUILDING
₹8,000
IN 22D
SP
Suresh Prabhu
SILVER OAK NEST · UNIT 3B
SCORE +11 · BUILDING
₹7,250
19D LATE

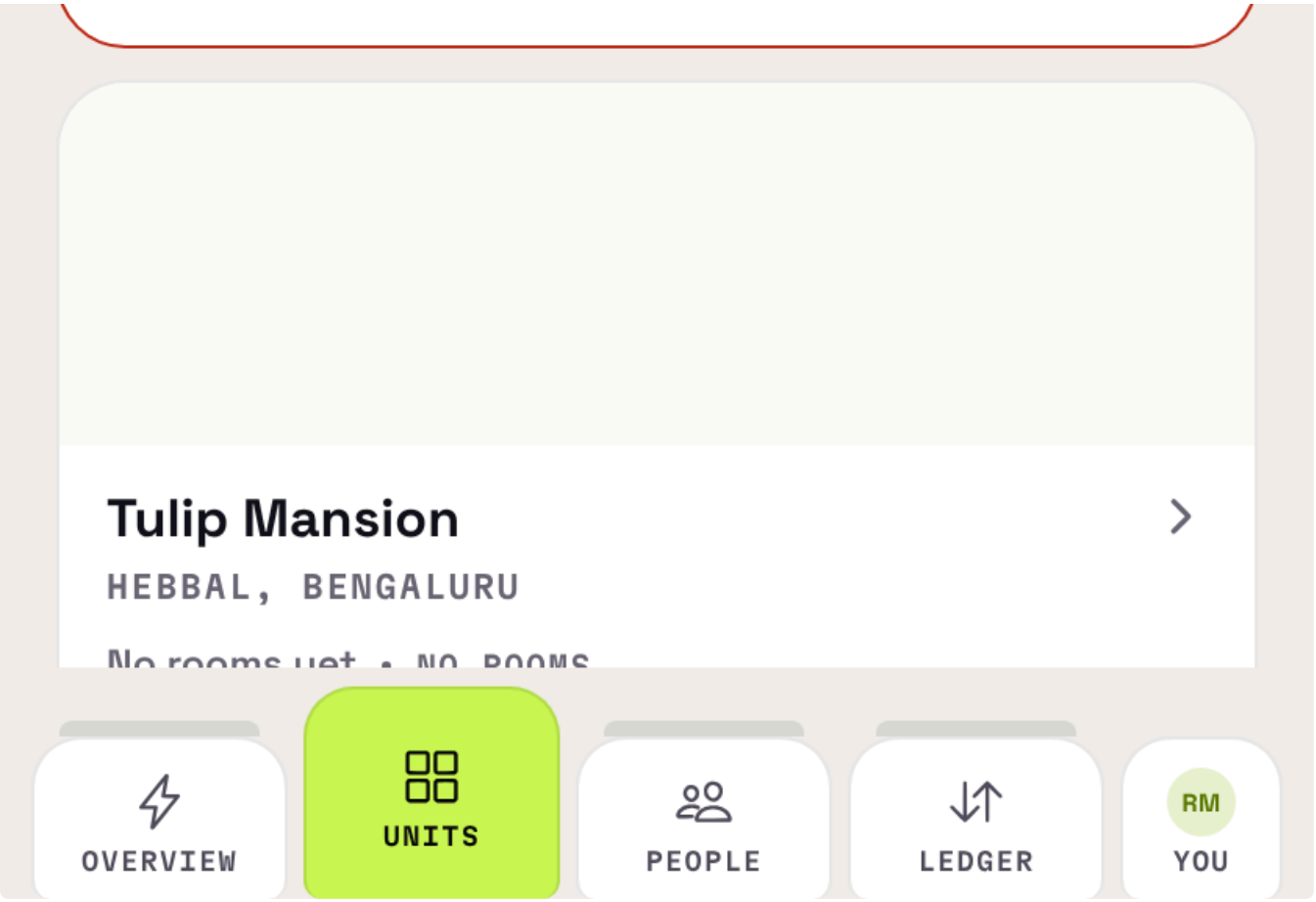



Each row carries the tenant's property and room, their rent share, a payment-behaviour score, and a due-state chip — `IN 7D`, `19D LATE`, or a leaving date for a tenant who has given notice. The filter chips are counted, so the distribution is visible before any of them is tapped.

`UNASSIGNED` is not an edge case bolted on: a tenant can exist without a room, and that state is reachable from the roster rather than hidden. The end-to-end suite asserts that such a row renders no broken room label.

## 3.3 Rooms and properties


Property, unit or tenant
2
Properties & units
3 properties · 6 rooms · 0 vacant
+ Add property
Palm Heights
BANASHANKARI, BENGALURU
Every room filled • 2 ROOMS
2 / 2 FULL
₹7,250 due · 3B
Silver Oak Nest
RAJAJINAGAR, BENGALURU
Every room filled • 4 ROOMS
4 / 4 FULL




Properties hold rooms; rooms hold tenants, with per-room rent, capacity and an equal or custom split across occupants. A room's status follows its occupancy automatically. Each room carries its own notification preferences — email, SMS and WhatsApp independently — because a landlord's channel choice is not uniform across a portfolio.

## 3.4 Ledger

# Ledger

Every rupee in and out · ALL PROPERTIES

NET — OCTOBER

**₹48,500**

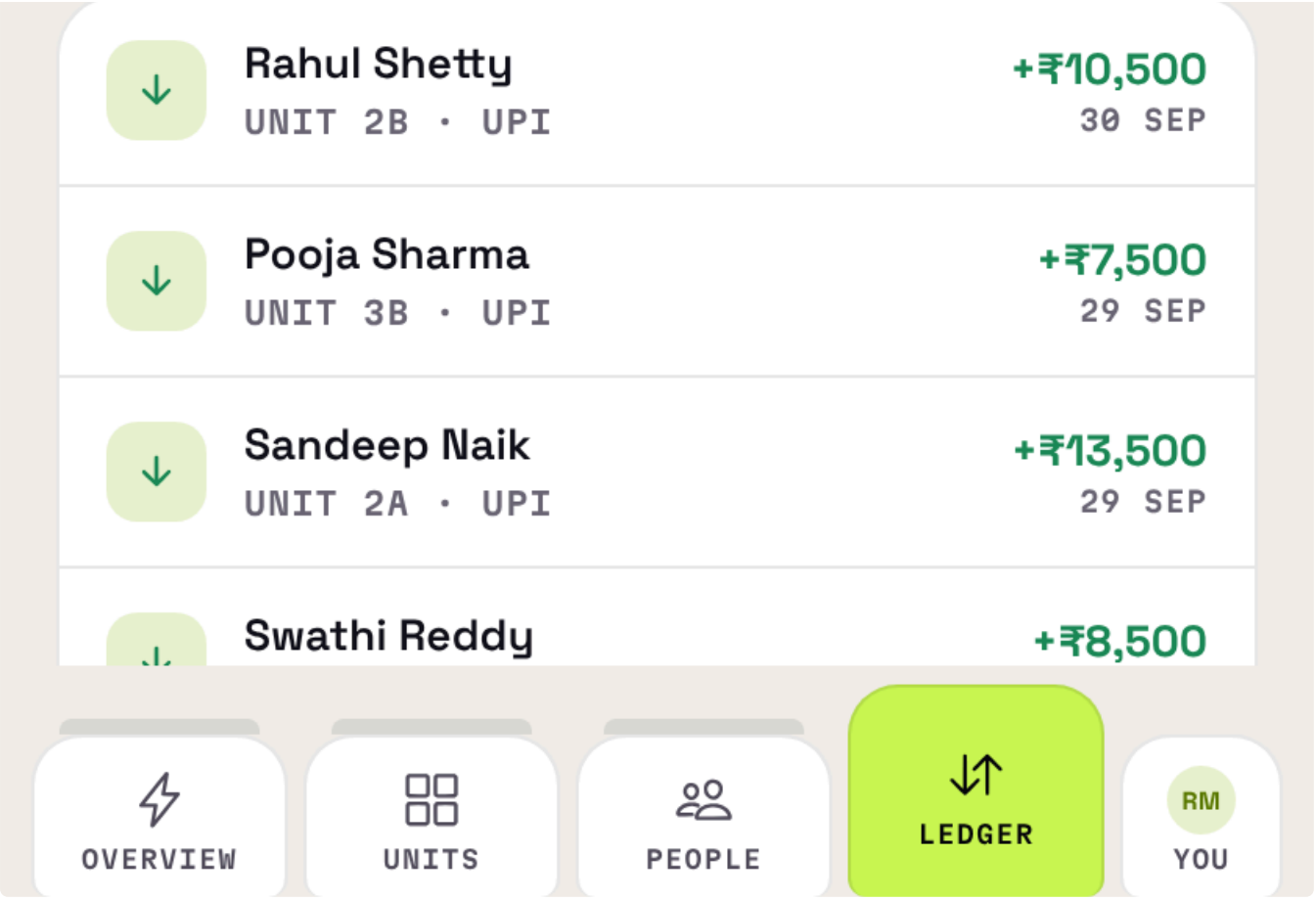
IN ₹48,500
OUT ₹0

Search tenant, unit or UTR

OCTOBER 2026 +₹48,500

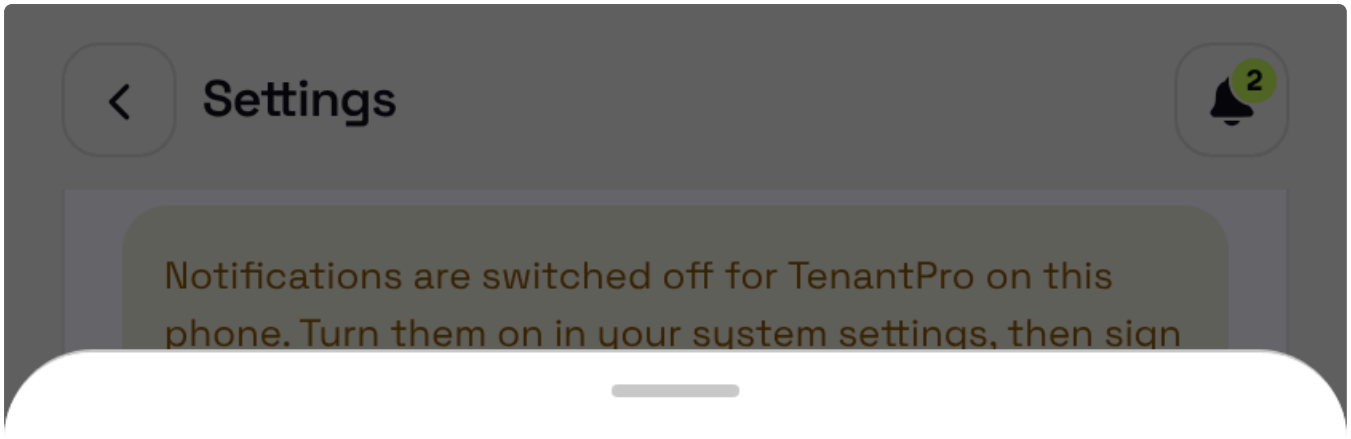
| | | |
|---|---|---|
| Akash Shetty<br>UNIT 2C · UPI | +₹16,500<br>4 OCT |
| Suresh Prabhu<br>UNIT 3B · CASH | +₹7,250<br>2 OCT |
| Deepa Shetty<br>UNIT 2A · BANK TRANSFER | +₹8,000<br>2 OCT |
| Ritu Menon<br>UNIT 2A · CASH | +₹8,500<br>1 OCT |
| Kavya Prabhu<br>UNIT 1D · UPI | +₹8,250<br>1 OCT |

SEPTEMBER 2026 +₹72,750




Every confirmed payment, searchable and scopable to one property. Payments carry the due date they settled, captured at declaration — without it, "was this on time?" is unanswerable after the fact, because the tenant's next-due date moves forward on confirmation and overwrites the only evidence. That omission was the reason the behaviour score once reported "None missed" for every tenant.

## 3.5 Connecting a payment gateway




# Online payments

Tenants can pay rent by card, netbanking or any UPI app, and it is confirmed automatically — you will not have to tap Confirm.

**WHAT IT COSTS**

The account is yours, so rent lands in it directly. Razorpay takes its usual fee (about 2% + GST) from each payment — this app charges you nothing, and your tenants pay only the rent.

Your UPI details stay exactly as they are. Tenants keep the free UPI option either way.

**BEFORE YOU START**

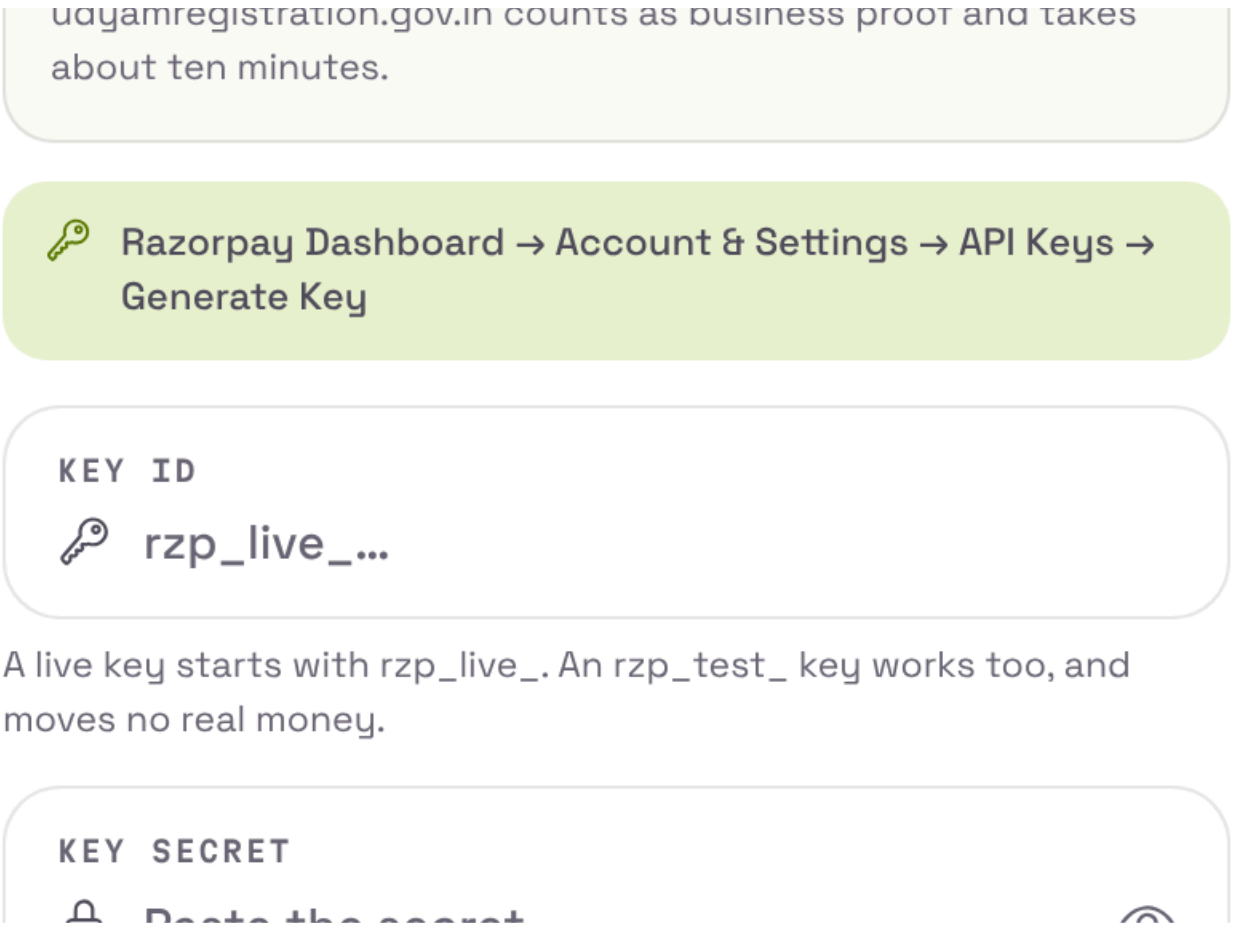
Razorpay will ask for your PAN, a bank account, and one business proof.

Live keys usually take 2–5 working days — KYC first, then a separate check of the website you give them.

---

✓ Test keys work straight away. Your tenants can keep paying by UPI the whole time.

No GST? A free Udyam (MSME) registration at




The sheet states the price next to the proposition, because quoting one without the other is how somebody enables a 2% fee without noticing. It also states what the gateway will demand before the landlord sets off — PAN, a bank account, one business proof, and two to five working days — along with the fact that test keys work immediately and tenants can keep paying by UPI throughout. A landlord expecting five minutes and meeting a week concludes the application is broken.

Credentials are verified against the gateway *before* being stored, so a mistyped secret is caught while the landlord is still looking at the form. They are then sealed with AES-256-GCM and never returned to any client.

### 3.6 Other implemented landlord features

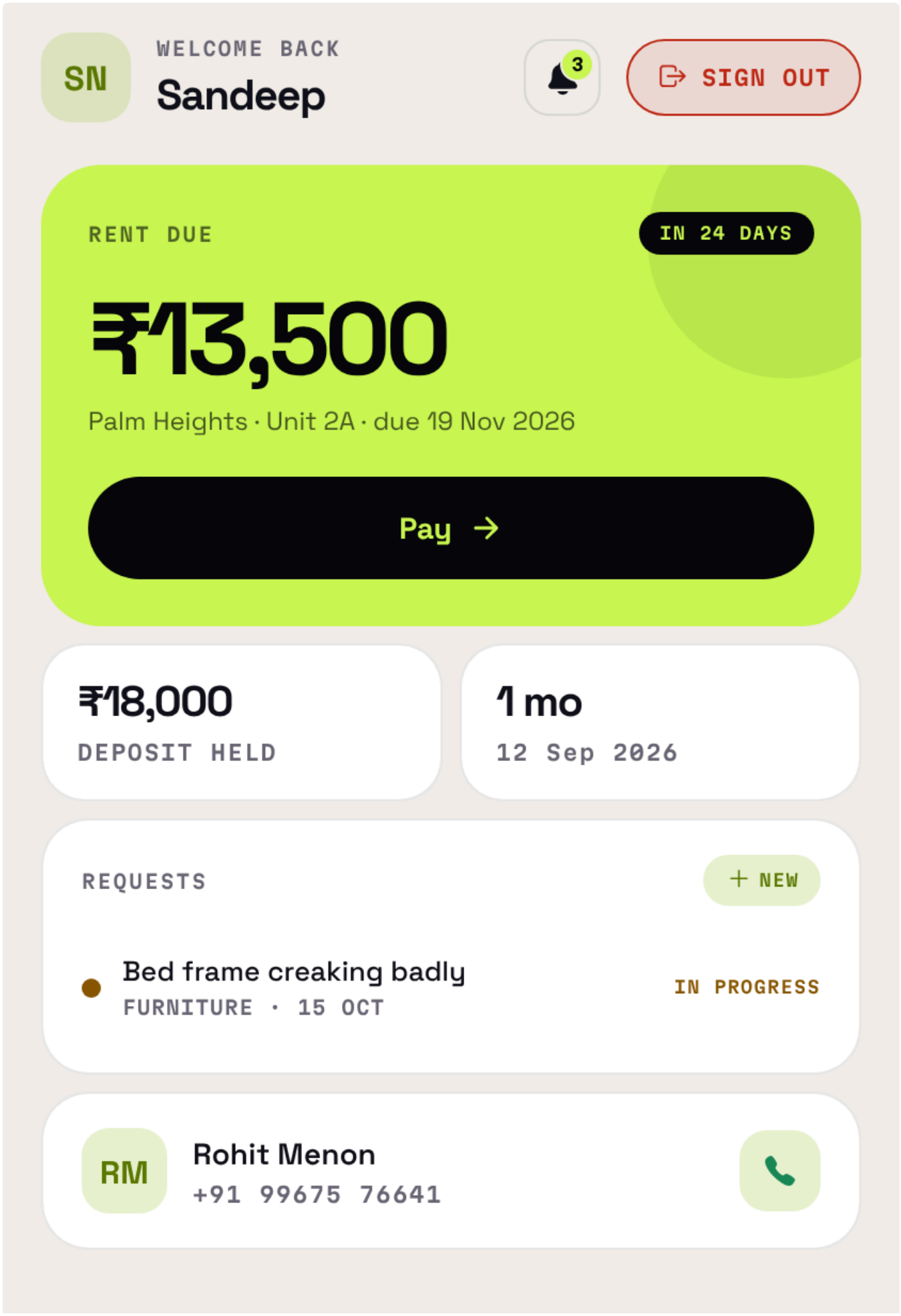
- **Join requests** — a tenant asks to join a property by code; the landlord accepts or declines, optionally assigning a room.
- **Maintenance threads** — per-request conversations shared with the tenant, with status transitions recorded as events in the thread.
- **ID documents** — tenant-uploaded proofs, with a blurred preview served under a short-lived token for tenants who have moved out.
- **Payment confirmation queue** — declared payments with the full context needed to decide, confirmed or rejected with a reason the tenant sees.

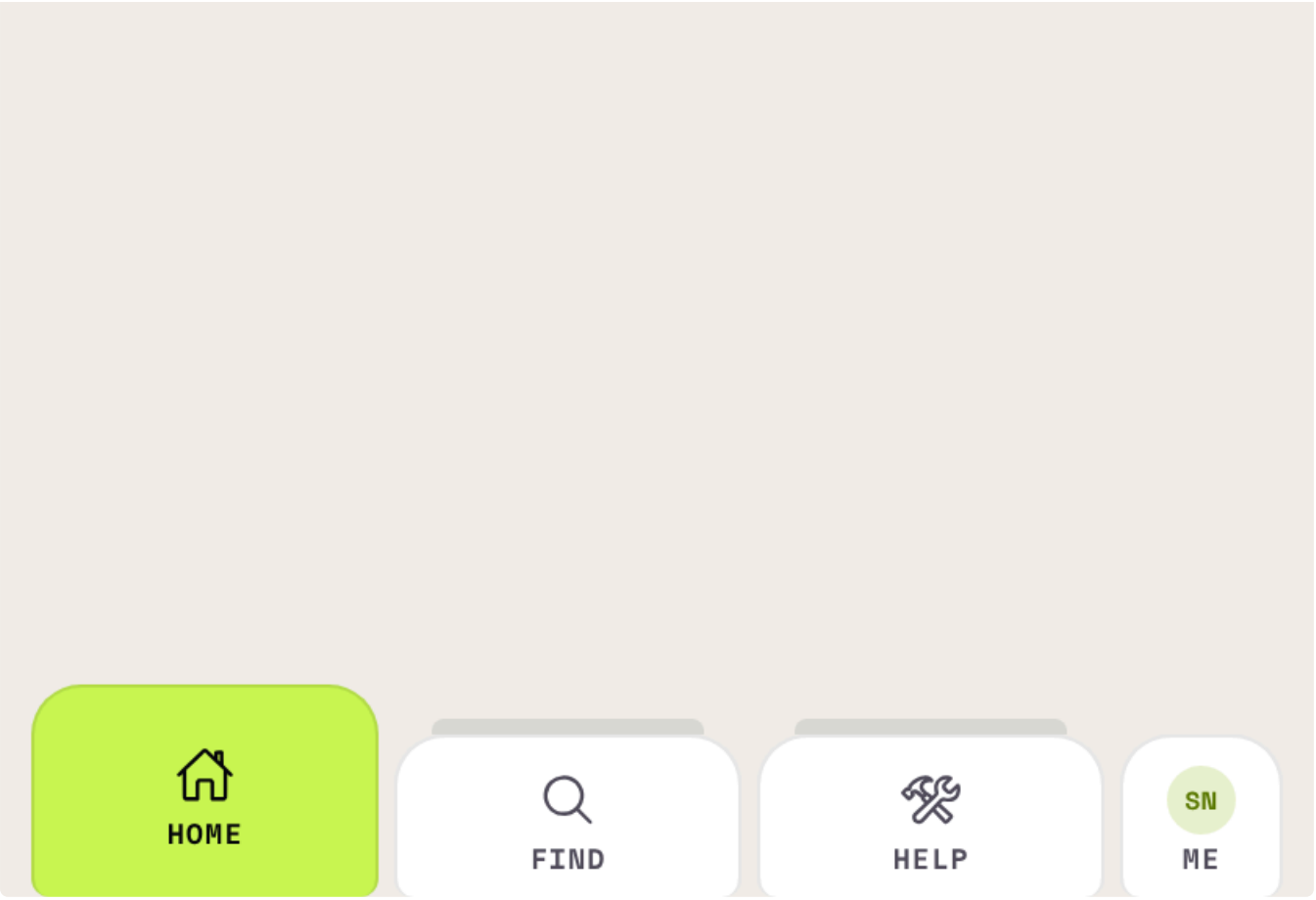
- **Notice periods** — a tenant's leave date, computed server-side and shown on both sides.
- **Daily rent reminders** — an 8am job across email, push, SMS and WhatsApp, each phrasing the same day identically.
- **Google sign-in**, profile management with contact verification, and a demo-account reset.

## 4. Tenant-side implementation

The tenant portal is not a reduced copy of the landlord application. It answers a different question — what do I owe, and has it been acknowledged — and it is reached by a separate account type with its own table and its own token role.

## 4.1 Home

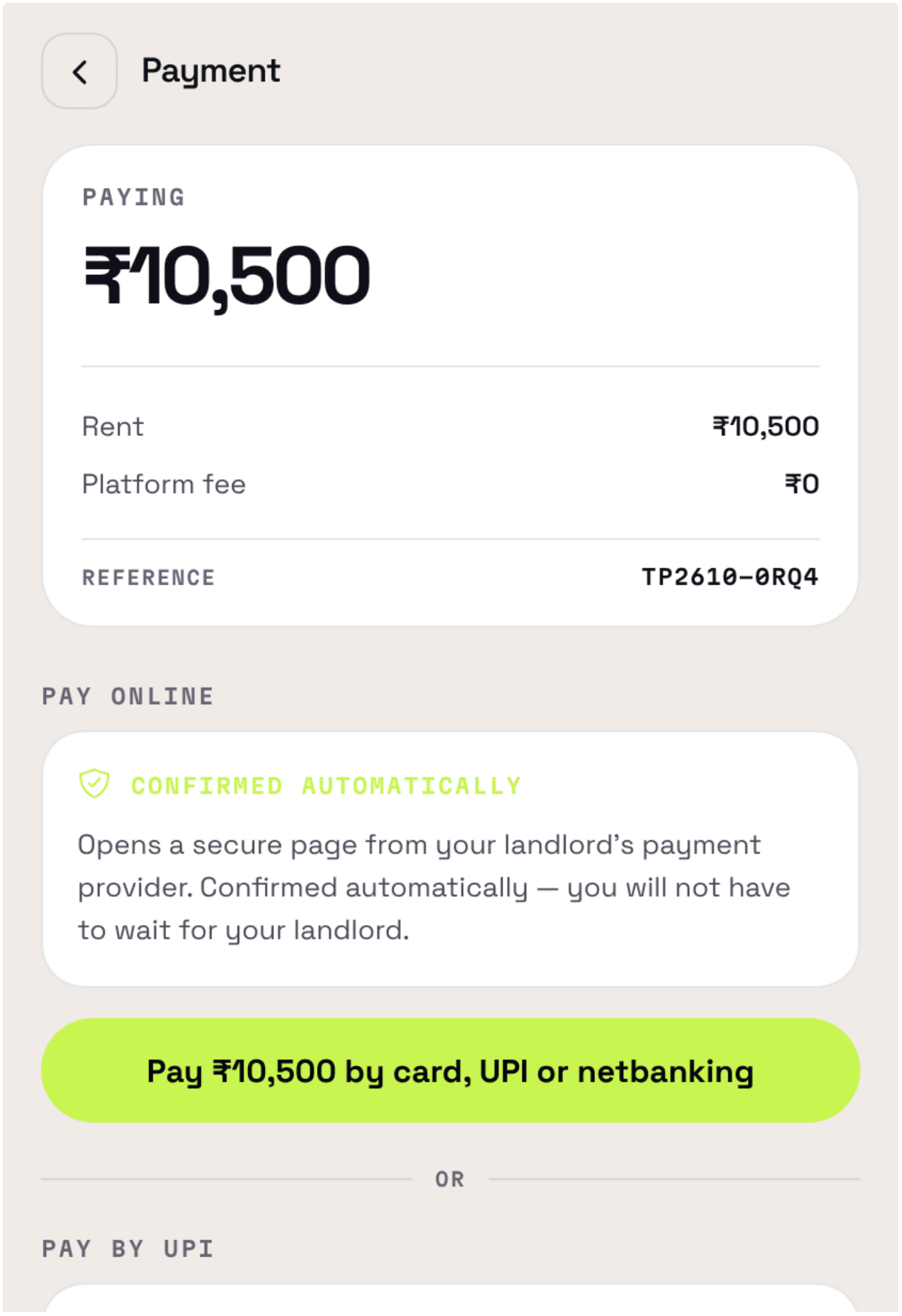

SN
WELCOME BACK
Sandeep
3
SIGN OUT
RENT DUE
IN 24 DAYS
₹13,500
Palm Heights · Unit 2A · due 19 Nov 2026
Pay →
₹18,000
DEPOSIT HELD
1 mo
12 Sep 2026
REQUESTS
+ NEW
Bed frame creaking badly
FURNITURE · 15 OCT
IN PROGRESS
RM
Rohit Menon
+91 99675 76641

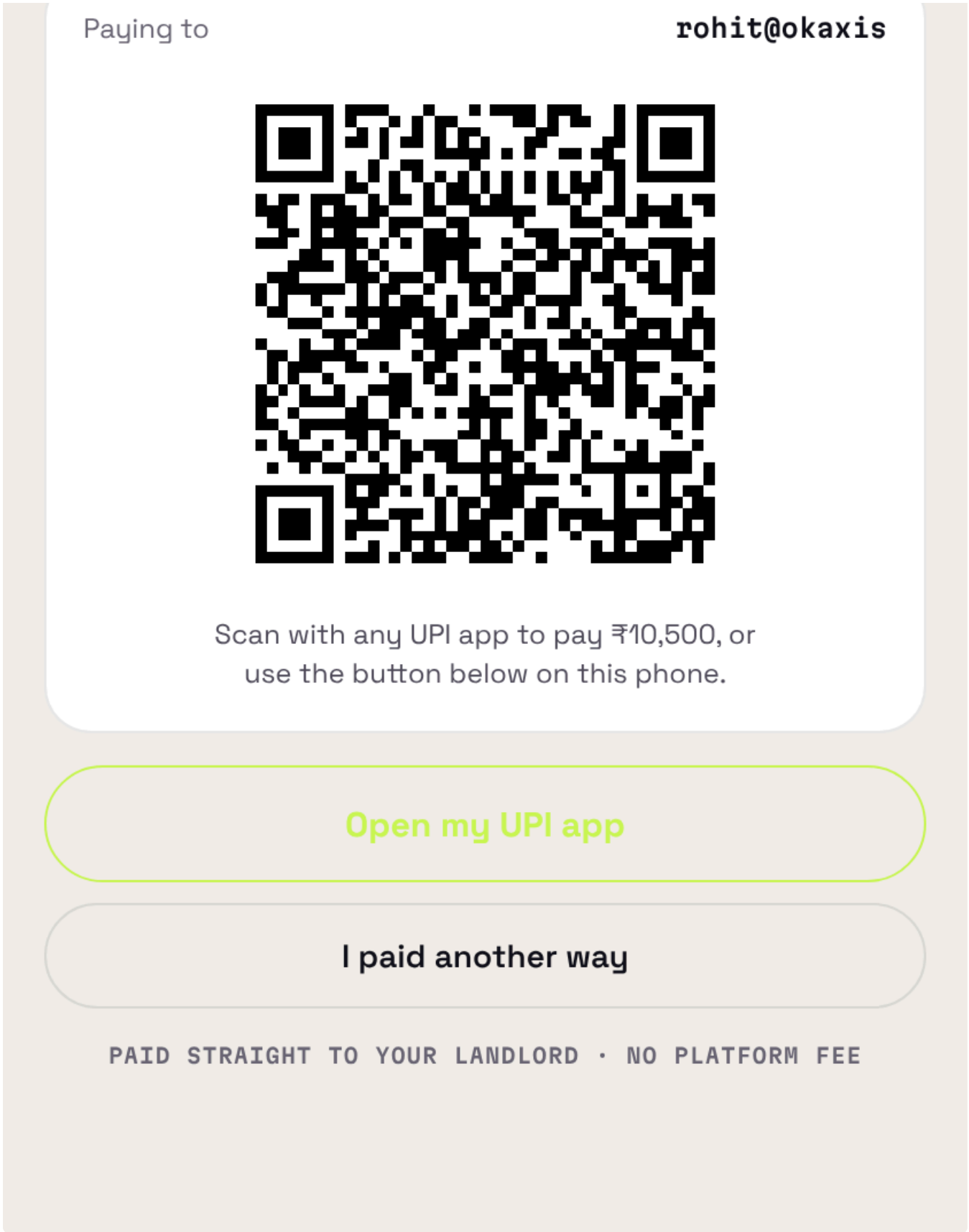



The rent card carries the amount, the property and room, the exact due date and a countdown. Deposit held and tenure sit beneath it, because both are facts a tenant routinely cannot recall and has no other record of.

The bell derives its state from the same payload the screen renders, rather than from a separate unread count that could drift. It surfaces rent falling due, a landlord's reply on a maintenance thread, a payment confirmed or rejected, and the outcome of a join request. A server-stamped `alerts_seen_at` distinguishes an alert that is *news* from one that is merely still true — and the server stamps it, so a device with a wrong clock cannot silence its own alerts.

## 4.2 Paying rent


Payment
PAYING
₹10,500
Rent
₹10,500
Platform fee
₹0
REFERENCE
TP2610-0RQ4
PAY ONLINE
CONFIRMED AUTOMATICALLY
Opens a secure page from your landlord's payment provider. Confirmed automatically — you will not have to wait for your landlord.
Pay ₹10,500 by card, UPI or netbanking
OR
PAY BY UPI




The screen shows what is owed, a reference, and ₹0 as the platform fee, which is literally true on both routes.

When the landlord has connected a gateway, **Pay by card, UPI or netbanking** appears first, because it is the only route that confirms itself. Beneath an `OR` divider sits the UPI route: the landlord's VPA, the amount and the reference encoded as a QR that works when scanned from another phone, plus a hand-off to the tenant's own UPI app on this one.

The two buttons are styled differently on purpose. Two filled primary buttons are two primary actions, which is none; the UPI button is outlined when a louder option sits above it and filled when it is the only option.

After returning from a gateway page the button reads **Check my payment**, not "Yes, I paid" — because on that route the answer is not the tenant's to give.

## 4.3 Profile and payment history

# My profile

SN

## Sandeep

UNIT 2A · Studio

12 Sep 2026

1 mo
WITH YOU

₹13,500
RENT SHARE

₹18,000
DEPOSIT

+26
CREDIT

YOUR CREDIT SCORE

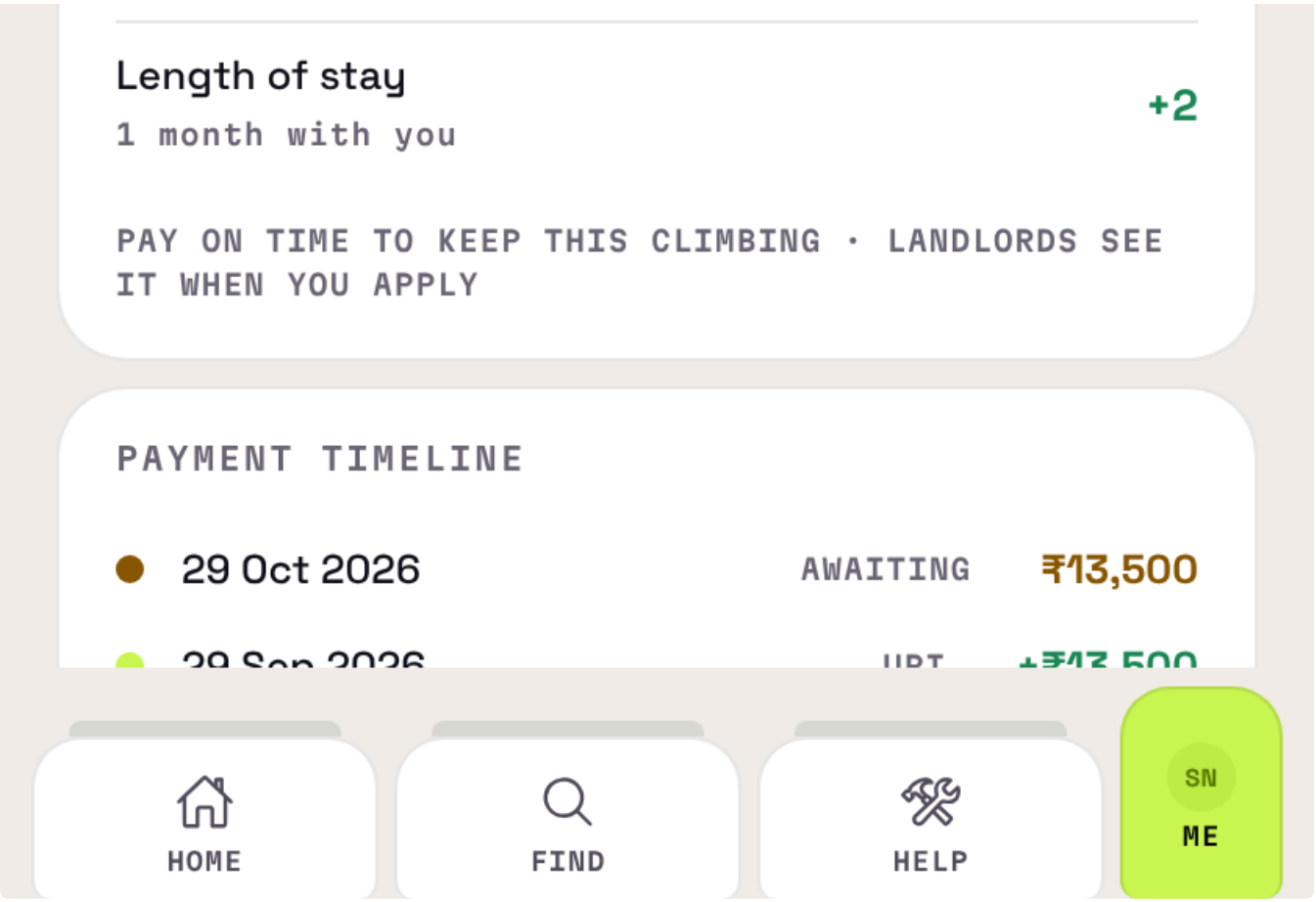
+26 Building

−100 NEGATIVE POSITIVE +100

Paid on time
4 payments on time
+24

Paid late
Never late
0

Currently overdue
Nothing pending
0




The behaviour score is computed by the same code the landlord's copy uses. Sharing it is the point: a tenant shown a different number from the one their landlord sees has no way to argue with either. Its factors are itemised rather than reduced to a dial, and when there is too little history the screen says so instead of parking a needle at zero, which would read as a verdict the tenant had earned.

The payment timeline is derived from actual rows. A claim still awaiting the landlord shows amber `AWAITING` rather than the green of money that has arrived — showing them alike is how a tenant comes to believe their month is clear when it is not.

## 4.4 Maintenance

# Help

+ Raise a ticket

Report an issue and follow it through.

**Bed frame creaking badly**

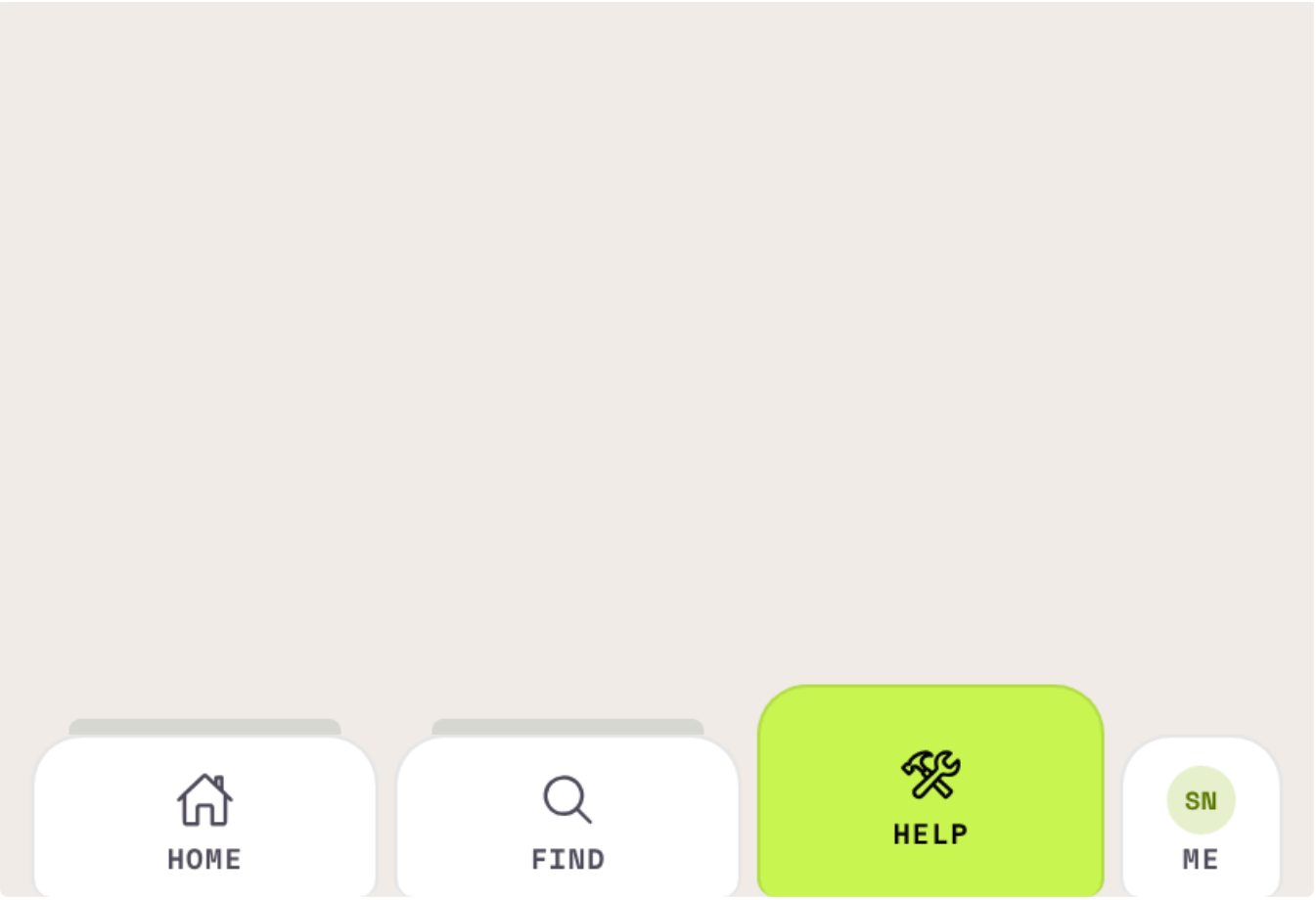
FURNITURE · 15 OCT

IN PROGRESS

RM

**Rohit Menon**

+91 99675 76641




A tenant raises a request with a category, a priority and an optional photo, then follows it through a thread shared with the landlord. Status changes appear in the thread as events, in sequence with what was said around them.

## 4.5 Other implemented tenant features

- **Guest accounts** — a tenant with no email, identified by a code, with a stay window; expiry is enforced server-side and the lock-out message offers the one action that resolves it.
- **Claiming an account** — a guest converts to a full account with a name, email and password.
- **Joining a property** — by scanning or typing an invite code, with the property shown before the request is sent.
- **Receipts** — the full payment history with status, reference, and the rejection reason where one was given.
- **Giving notice** — with the real dates computed and shown before anything is written, and withdrawable afterwards.
- **ID documents** — upload and removal, with a summary the landlord sees when deciding on a join request.
- **Directions** — to the tenant's own property, when the landlord has pinned it.

# 5. The payment and settlement model

Three structures exist for moving money between two parties on a platform. TenantPro occupies the third, and the choice is deliberate rather than incidental.

| Model | Who holds the funds | Authorisation required | Example |
|---|---|---|---|
| Aggregator | the platform, in a ring-fenced escrow at a scheduled commercial bank | RBI payment-aggregator licence | Uber, Ola |
| Split settlement | the gateway, settling to linked sub-merchant accounts | held by the gateway, not the platform | Razorpay Route |
| Direct | nobody — payer pays payee | none | Namma Yatri, **TenantPro** |

Under RBI rules an aggregator may not hold customer funds in its own account; the money sits in a segregated escrow that cannot earn interest for the platform and must be settled out on a deadline. Obtaining that authorisation requires a net-worth floor in the tens of crores. TenantPro avoids the question entirely by never receiving the money.

## 5.1 Two payment routes, both direct

**UPI.** The tenant's screen carries the landlord's own VPA, the exact amount and a generated reference, encoded as a QR and as a `upi://` intent. The tenant pays from their own UPI app. Cost to everyone: zero.

**Gateway.** A landlord may connect their own Razorpay merchant account. The server then creates a Payment Link against *their* credentials; the tenant pays on the gateway's hosted page and the money settles to that landlord. Razorpay's fee is charged by their gateway to them. The platform bills nothing because it is party to nothing.

Payment Links were chosen over Razorpay's React Native SDK specifically because the SDK is a native module. A link is plain REST over HTTPS, so the payment feature ships over the air like every other change.

## 5.2 The confirmation model

A payment begins life as a `Declared` row — a claim, which clears nothing. It becomes real when a second party vouches for it:

- the **landlord** taps Confirm, or

- the **gateway** is asked directly, server-to-server, using the landlord's credentials, and answers that the link was paid.

Both paths call one function, `settlePayment`, which flips the status and advances the tenant's next due date inside a single transaction. Two implementations would eventually disagree about when a month clears, so there is one.

The gateway is polled rather than relied on to call back. A webhook requires each landlord to configure a URL and a secret in a dashboard, and a feature gated on that step is one most landlords would never finish switching on. The signed-webhook path exists and remains supported, but it is the secondary route.

## 6. Authentication and session management

The session design went through three states, and the progression is instructive because the first two each fixed a real problem while introducing the next one.

| Stage | Access token | Can be revoked | Failure it solved | Failure it left |
|---|---|---|---|---|
| Original | 7 days, fixed | no | — | weekly logout for every user |
| Sliding | 30 days, renewed on use | no | the weekly logout | nothing could be cancelled |
| Current | 2 hours + refresh token | yes | revocation | instant revocation on the access path |

### 6.1 Why a longer token was not the fix

Raising the lifetime from 7 days to 30 did not stop people being logged out; it made it monthly. A fixed lifetime measures from the last *sign-in*, which says nothing about whether somebody is still using the application. Someone who opened the app every morning was ejected on day 30 exactly like someone who had not opened it since February.

The server therefore re-issues the access token on any authenticated request once it is past half its life, returning it on an `X-Session-Token` header the client adopts. The new token's lifetime is read from the old one's `exp - iat` rather than a constant, so a guest session keeps its own length and legacy tokens keep theirs.

### 6.2 Refresh tokens and revocation

An access token is a signature and nothing else: the server consults no list, which is what makes it fast and also meant nothing could withdraw it. A stolen phone, a departed tenant, a password change — none ended a session.

A session is now also a row holding a refresh token, stored as a SHA-256 hash rather than the token itself. The design has three properties worth naming:

1. **Rotation.** Spending a refresh token returns a replacement. A sixty-day secret crossing the network on every reconnect is a long-lived credential; a single-use one is not.
2. **Reuse detection.** Because rotation means the honest client always holds the newest token, replaying a spent one implies a copy exists elsewhere. That revokes every session on the account. A false positive costs one sign-in; a false negative costs the account.
3. **Revocation on password reset.** Resetting a password now ends every other session, which is the thing users already believe it does.

Revocation is immediate on the refresh path and lags by at most the access token's two-hour life on the access path. Making it instant would require a database read on every authenticated request — a cost almost no system pays. Two hours against the previous thirty days is the improvement.

### 6.3 Client-side failure handling

The client half is where this design fails badly if it fails at all. A 401 triggers one refresh and one replay, and only a 401 *from the refresh endpoint itself* ends the session; a 500, a timeout or a dead connection does not. The server deliberately answers 500 rather than 401 on its own faults so the client can tell them apart.

The sharpest failure mode was concurrency. An application firing eight parallel requests on a cold start, all failing on the same expired token, would otherwise issue eight refreshes — seven of them replaying a token the first had already spent, which the server reads as theft and answers by ending every session. The feature intended to prevent lockouts would have caused them at scale. One shared in-flight promise prevents it, and the behaviour is asserted directly.

## 7. Verification

The build is verified by 386 assertions across six suites. None uses a test framework; each is a plain Node script that exits non-zero on failure.

| Suite | What it drives | Isolation | Assertions |
|---|---|---|---|
| `secretbox` | AES-256-GCM sealing of gateway credentials | pure | 24 |
| `session` | token renewal, through the real middleware and the real interceptor | stubbed db | 41 |
| `gateway` | gateway controllers | stubbed db and `fetch` | 72 |
| `vm.gateway` | the real `deriveVm`, react-native stubbed | stubbed native | 52 |
| `refresh.client` | the real axios interceptor: refresh, retry, and when not to sign out | stub adapter | 29 |
| `refresh.e2e` | refresh tokens, reading the database directly | live server + MySQL | 42 |
| `api.e2e` | all 87 endpoints | live server + MySQL | 126 |
| `ui.e2e` | the compiled application in a browser engine | live stack + Chromium | 71 |

## 7.1 Seeded fixtures

A deterministic generator ( `scripts/seed.js` ) produces a portfolio that includes, on purpose, the rows that break screens and never arise by accident: a tenant with no room, a guest with no email, a guest whose stay has *already* ended, rent overdue and due-today and upcoming and never set, a payment declared and undecided, one rejected with a reason, maintenance threads where the landlord spoke last and where the tenant did, a tenant who has given notice, and a property with no rooms. A seeded PRNG makes any failing run reproducible.

## 7.2 What the browser suite does and does not cover

The UI suite runs the same JavaScript the phone runs — the same view-model, screens, API client and interceptors — laid out by `react-native-web`, which is what an Expo web export produces. It cannot cover anything behind a native module: camera, image picker, push delivery, the UPI intent hand-off. It prints `SKIP` rather than a pass for anything it could not reach, so a green run never implies coverage it does not have.

### 7.3 Defects found by the end-to-end runs

Driving every endpoint against a real database, and the application against a real browser, surfaced five defects that unit tests had not.

| Defect | Severity | Status |
|---|---|---|
| Two of three endpoints accepting a `unit_id` checked ownership; the third did not, allowing one landlord's tenant to be filed into another's room | cross-account read | fixed — one shared check |
| A wrong password rendered "Your session expired", overriding the server's own `WRONG_PASSWORD` message | user-facing | fixed |
| Five required-field omissions produced HTTP 500 with a raw SQL error instead of a 400 | robustness | fixed |
| Renaming a property without its address crashed: `NOT NULL` column, coerced on insert but not on update | robustness | fixed |
| A network failure reaching the gateway was reported as a rejected credential | user-facing | fixed — transport failures now distinguished |

Two further defects were found **in the test harnesses themselves**, which is worth recording. The first browser run never cleared the permissions primer, and because its assertions were phrased as the *absence* of a string the primer also lacked, half the suite passed while testing nothing. Separately, the failure reporter crashed on a failure whose context was `undefined`, taking down the run it was trying to report. Both argue for positive assertions and for a failure reporter being the dullest code in the file.

## 8. Limitations

Stated plainly, because each is a consequence of a design decision rather than an oversight.

**The application cannot observe money.** It never sees a bank balance, so a UPI payment is a claim until the landlord confirms it. The gateway route removes this for landlords who connect one; for everyone else the confirmation step is structural.

**Revocation lags by up to two hours on the access path.** Closing that gap entirely means a database lookup on every authenticated request.

**Reuse detection can produce a false positive.** A client that stores a new access token but fails to store its paired refresh token will present a spent token and have every session revoked. The write is transactional on the client side, but a device that loses storage mid-write would hit this.

**SMS is unconfigured.** Phone verification requires DLT registration with an Indian telecom operator, which has not been completed. Email and push are live.

**iOS has never been built.** `ios.bundleIdentifier` is unset; the project has only ever produced Android artifacts.

**Gateway onboarding is heavy.** A landlord connecting Razorpay faces their KYC and a separate website verification — realistically two to five working days. The in-app sheet now states this before they begin, but it remains the single largest piece of friction in the product.

**No automated regression gate.** The suites exist and pass, but nothing runs them on push; verification is currently manual and deliberate.

# 9. Future scope

## 9.1 Identified and specified

**Reply notifications on maintenance threads.** A tenant replying to a repair request currently notifies nobody: `createRequestMessage` issues no `notify` call and the push rules carry no owner-facing kind for a reply. The landlord-to-tenant direction already works.

**A transaction around `recordPayment`.** The landlord's own entry path inserts the payment and updates the due date as two separate statements. The tenant-facing `settlePayment` is already transactional; this path is not, and a failure between the two leaves a payment recorded against a date that never moved.

**An automated test gate.** The suites are reproducible and fast enough to run on every push; wiring them to CI is a configuration task, not a development one.

**iOS.** A bundle identifier and a first build.

## 9.2 Larger directions

**Split settlement, when there are landlords other than the author.** Razorpay Route would let landlords onboard as linked sub-merchants inside the application, with the gateway performing their KYC — removing the onboarding friction named in Section 8. The cost is

that funds would pass through a platform account, which is a category change rather than a configuration one, and the 2025 RBI directions restrict split settlement in ways that need professional advice rather than inference. Worth revisiting when the user base justifies it; not before.

**Monetisation without entering the money path.** The Namma Yatri precedent is directly applicable: a flat per-landlord subscription keeps the operator outside the payment flow entirely, where a per-transaction percentage structurally resembles aggregation. A related legal nuance is that such models rest on selling software access and not setting the price — and in TenantPro the landlord sets the rent, which is a position worth preserving deliberately.

**Deliverability.** Transactional mail currently sends from a Gmail address through Brevo's HTTPS API. A verified sending domain with SPF, DKIM and DMARC would materially improve inbox placement, at roughly ₹1,000 per year.

**SMS and DLT.** Phone verification needs an Indian DLT registration before an operator will carry template traffic.

**Landlord announcements.** A one-to-many message to a property's tenants ("water off Tuesday") was specified during the notification work and never built; the alert infrastructure to carry it already exists.

## 10. References

Code figures, endpoint counts and assertion totals in this report were measured directly from the repository at commit `f065250` and from live test runs, not estimated.

**Regulatory and payment-model sources**


- Nodal and escrow accounts under RBI rules — how aggregators are required to hold customer funds.
- RBI rewrites the payment aggregator rulebook — commentary on the Regulation of Payment Aggregators Directions, 2025.
- Razorpay Route — split settlement and linked accounts.
- Razorpay API keys — key generation, and the website verification required for live mode.
- Razorpay KYC documents by business type.
- Namma Yatri's zero-commission model — the direct-payment precedent discussed in Section 5.


**Note on secondary sources.** Several regulatory points above rest on consultancy and industry commentary rather than the RBI's own text. Where this report draws on them — particularly the net-worth thresholds and the restrictions on split settlement — the

claims should be verified against the Directions themselves before any structuring decision is taken.